

Projet RCP209

Apprentissage statistique 2

Grossi Valério

7 Septembre 2025
2024-2025 Semestre 2

Sommaire

- 1. Introduction**
- 2. Présentation de l'étude**
- 3. Solution proposée**
 - 3.1 Sélection des variables
 - 3.2 Hyperparamètres et familles paramétriques testés
- 4. Implémentation**
- 5. Résultats**
 - 5.1 Classifieur optimal sur les six variables principales
 - 5.2 Classifieur optimal sur les variables libres
- 6. Conclusion**

1. Introduction

Dans le cadre du module *Apprentissage statistique 2 RCP209*, des méthodes d'apprentissage supervisé dans le cadre de la modélisation décisionnelle sont enseignées. Le projet traité est « Human Activity Recognition » et a pour objectif d'implémenter ces notions au travers de la création d'un classifieur de types d'activités physiques humaines à partir de séries temporelles multivariées collectées à partir de données IMU de smartphones.

L'étude et ses données d'entrées sont d'abord présentées. L'approche de résolution employée ainsi que son implémentation pratique sont ensuite exposées. Enfin, les résultats obtenus sont discutés.

2. Présentation de l'étude

L'objectif des sujets assignés au thème « Human Activity Recognition » est la réalisation de classifieurs ayant pour fonction la reconnaissance de types d'activités physiques humaines au sens large.

Ces classifieurs sont construits le plus souvent par apprentissage supervisé. Des données collectées et bien étiquetées sont alimentées au classifieur pour son entraînement.

Dans notre sujet, la tâche de classification consiste à sélectionner la bonne activité physique parmi les six types d'activités possibles en entrée, qui sont « être allongé », « être assis », « être debout », « marcher », « descendre des escaliers » et « monter des escaliers ».

Les données d'entrées sont des séries temporelles multivariées (jusqu'à 562 variables bien définies) collectées à partir de données IMU de smartphones (<https://www.kaggle.com/datasets/uciml/human-activity-recognition-with-smartphones>).

Pour l'apprentissage des classifieurs souhaités, des données d'entrées étiquetées du type d'activité correspondant sont mises à disposition. Ces données d'entrées sont séparées en deux groupes, un set d'entraînement composé de 280 séries temporelles, et un set de test composé de 120 séries temporelles.

L'objectif est donc d'entraîner plusieurs classifieurs sur la base des données mises à disposition et de comparer les résultats obtenus pour différentes métriques bien choisies afin d'en sélectionner le meilleur.

3. Solution proposée

Afin d'obtenir le meilleur classifieur possible, on propose d'en entraîner plusieurs, pour diverses familles paramétriques et hyperparamètres.

L'entraînement des différents classifieurs s'effectue sur la base des données du set d'entraînement uniquement. Le choix du meilleur classifieur s'effectue au regard des résultats obtenus sur leurs scores de validation, puis on donne également le score de test du meilleur classifieur retenu.

On propose ainsi pour la sélection du classifieur de ne pas se baser sur le score de test, obtenu sur les données de test, mais uniquement sur un score de validation, également obtenu sur des données inconnues mais via un processus de validation croisée lors de l'entraînement.

Également, il peut être intéressant d'effectuer un filtrage des variables d'entrées considérées pour le choix du meilleur classifieur. On propose ainsi deux classifieurs optimaux, l'un basé uniquement sur les 6 variables principales d'une IMU (accélérations linéaires et vitesses angulaires) et l'autre sur un certain nombre de variables bien choisies parmi les 562 mises à disposition (en réalité sur 561 variables car on élimine d'office la variable « subject » propre à la personne qui porte le smartphone).

3.1. Sélection des variables

Pour le choix du meilleur classifieur avec son groupe idéal de variables d'entrées à déterminer, on utilise deux méthodes : filtrage et sélection en avant.

La méthode de filtrage des variables est réalisée en amont de la sélection du meilleur jeu d'hyperparamètres, elle est indépendante de l'entraînement.

La méthode de sélection en avant peut-elle être intégrée au processus de recherche du meilleur jeu d'hyperparamètres, mais peut également être effectuée après avoir fixé un jeu d'hyperparamètres jugé satisfaisant. D'ailleurs, c'est cette seconde option qui a été retenue, en raison des contraintes de temps de simulation, bien que l'implémentation proposée permette également la première approche.

Concernant la méthode de filtrage des variables, pour chaque séquence, on calcule la moyenne et l'écart-type de chaque variable. On effectue ensuite sur chaque variable et pour chacune de ces deux statistiques un test non paramétrique de Kruskal-Wallis qui fournit un score de discrimination sur les classes. On sélectionne enfin les 15 meilleures variables pour chacune des deux statistiques, soit un maximum de 30 variables.

3.2. Hyperparamètres et familles paramétriques testés

Les hyperparamètres et familles paramétriques définis ici sont ceux de *Pytorch*. Les combinaisons des hyperparamètres et familles paramétriques testés sont sélectionnées par recherche aléatoire.

Hyperparamètres liés à la procédure :

- Métrique utilisée : [[CrossEntropyLoss](#)]
- Nombre de splits de la stratified k-fold cross validation [[10](#)]
- Graine aléatoire pour la reproductibilité des découpages de la validation croisée entre chaque jeu d'hyperparamètres [[42](#)]

Familles paramétriques :

- [LSTM](#)
- [CNN + LSTM](#)
- [Transformers](#)

Hyperparamètres architecturaux :

- [LSTM](#) :
 - Nombre de couches : [[2](#), [3](#), [4](#)]
 - Dimension de l'état caché : [[64](#), [80](#), [96](#)]
- [CNN + LSTM](#) :
 - Dimension de sortie produite par le CNN : [[8](#), [16](#), [32](#)]
 - Nombre de couches du LSTM : [[2](#), [3](#), [4](#)]
 - Dimension de l'état caché du LSTM : [[64](#), [80](#), [96](#)]
- [Transformer](#) :
 - Dimension d'entrée et de sortie de chaque couche : [[36](#), [48](#), [60](#), [72](#)]
 - Nombre de têtes d'attention : [[2](#), [3](#), [4](#), [12](#)]
 - Nombre de couches : [[2](#), [3](#), [4](#)]

Hyperparamètres d'optimisation :

- Optimiseur [[Adam](#)]
- Taux d'apprentissage [[3.5e-3](#), [4e-3](#), [4.5e-3](#), [5e-3](#), [5.5e-3](#)]
- Planificateur du taux d'apprentissage [[ReduceLROnPlateau](#)]
- Facteur de réduction du taux d'apprentissage [[0.4](#), [0.5](#), [0.6](#)]

- Nombre d'époques de tolérance du planificateur [30]
- Nombre de cycles d'entraînement maximal [500]
- Taille du lot [28, 32, 36]

Hyperparamètres de régularisation :

- Régularisation L2, facteur de décroissance des poids paramétré dans l'optimiseur [5e-7, 7.5e-7, 1e-6, 2.5e-6, 5e-6]
- Dropout [0, 0.025, 0.05, 0.075, 0.1, 0.125, 0.15, 0.175]
- Nombre de cycles d'entraînement maximal sans amélioration avant arrêt précoce [80]

4. Implémentation

```

└─ HumanActivityRecognition/
  │
  └─ logs/
    │
    └─ 2025-06-15_20-05-30/
      │
      └─ output.log
      │
      └─ resultats_modeles.csv
      │
      └─ resultats_modeles.png
      │
      └─ results.pkl
      │
      └─ confusion_matrix_idx.png
      │
      └─ ...
    │
    └─ Import_Data.py
    │
    └─ Variable_Selection.py
    │
    └─ preprocessing.py
    │
    └─ estimator.py
    │
    └─ PyTorch_cv.py
    │
    └─ analyse.py
  
```

Structure du projet

Import_Data.py a pour unique fonction le téléchargement des datasets train.csv et test.csv depuis le site Kaggle.

Variable_Selection.py définit la fonction qui prends en entrée un dataset au format csv et retourne la liste des noms des variables sélectionnées selon la procédure décrite au 3.1 (filtrage en amont des variables).

preprocessing.py définit la fonction qui prend en entrée un dataset au format csv ainsi qu'une liste de noms de variables et retourne en sortie les tenseurs X et y associés de formes $[n_sequences, max_len, n_features]$ et $[n_sequences]$.

Les classes 'LAYING', 'SITTING', 'STANDING', 'WALKING', 'WALKING_DOWNSTAIRS' et 'WALKING_UPSTAIRS' sont encodées par entiers. Les données sont normalisées (centrage-réduction) puis un padding de zéros est effectué afin d'uniformiser la longueur des séquences à *max_len*. La distribution des classes étant déjà équilibrée pour le dataset de train comme de test, aucune action de rééquilibrage n'a été appliquée en ce sens.

estimator.py définit les classes des familles paramétriques qui seront utilisées.

PyTorch_cv.py exécute les fonctions définies dans *Variable_Selection.py* et *preprocessing.py*, puis teste les valeurs des hyperparamètres et familles paramétriques qui y ont été renseignées. Le script sauvegarde tous les résultats dans une liste de dictionnaires sérialisée dans *results.pkl* et enregistre le déroulement des entraînements dans le fichier *output.log*.

analyse.py récupère les sorties des expériences contenues dans le fichier binaire *results.pkl*, calcule un score personnalisé sur les résultats des combinaisons testées, trie les résultats suivant ce score et les sauvegarde dans *resultats_modeles.csv*.

Un graphique des dix meilleures configurations est tracé et sauvegardé dans *resultats_modeles.png*. Le score considéré est la précision moyenne en validation diminuée de son écart-type.

Le script permet également de tracer et sauvegarder dans *confusion_matrix_idx.png* la matrice de confusion sur les données de test pour la combinaison d'index *idx* renseignée (par défaut, c'est l'index correspondant au meilleur résultat obtenu selon le score personnalisé qui est choisi).

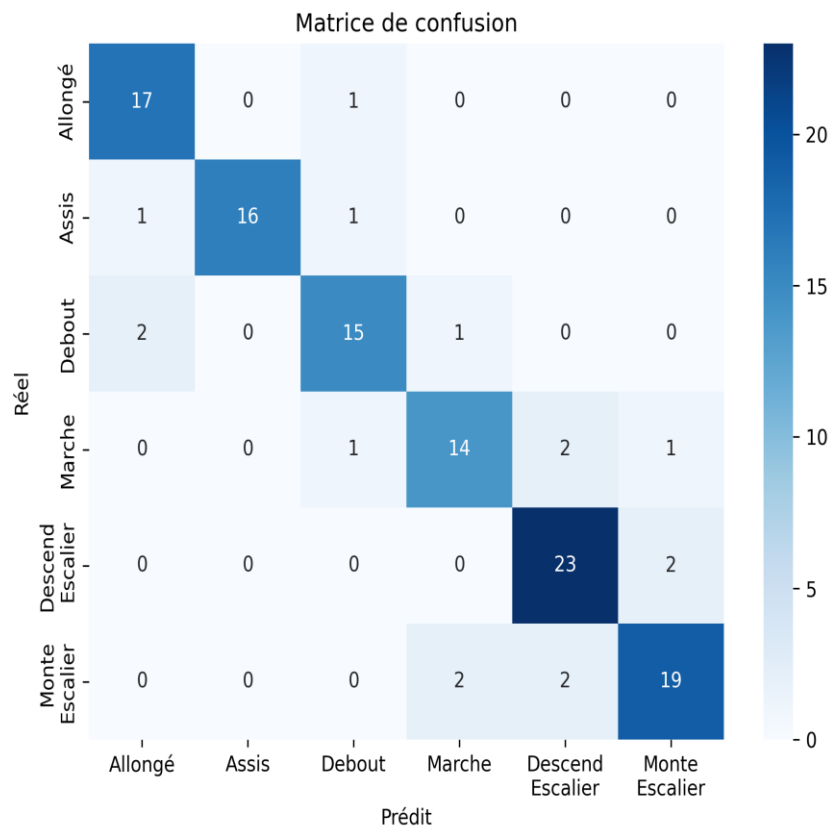
5. Résultats

5.1 Classifieur optimal sur les six variables principales

Concernant la sélection du classifieur optimal nécessitant en entrée les six variables principales d'une IMU (accélérations linéaires et vitesses angulaires), plus précisément *tBodyAcc-mean()-X*, *tBodyAcc-mean()-Y*, *tBodyAcc-mean()-Z*, *tBodyGyro-mean()-X*, *tBodyGyro-mean()-Y* et *tBodyGyro-mean()-Z*, les résultats sont contenus dans *logs/2025-08-29_17-30-59* et *logs/2025-09-01_19-14-30*.

Lors de la première exécution, 100 jeux d'hyperparamètres parmi ceux présentés au 3.2 ont été testés. Les résultats obtenus sur les jeux ayant comme famille paramétrique les Transformers se sont avérés, en moyenne, nettement supérieurs à ceux ayant comme famille paramétrique CNN + LSTM et LSTM.

Ainsi, lors de la deuxième exécution, il a été joué 300 jeux d'hyperparamètres parmi ceux présentés au 3.2 mais en restreignant le choix de la famille paramétrique à celui des Transformers uniquement. On obtient alors comme meilleur résultat un classifieur ayant pour score de validation 96.8%+-1.9% et pour score de test 86.7%.



Matrice de confusion obtenue sur les données de test

5.2 Classifieur optimal sur les variables libres

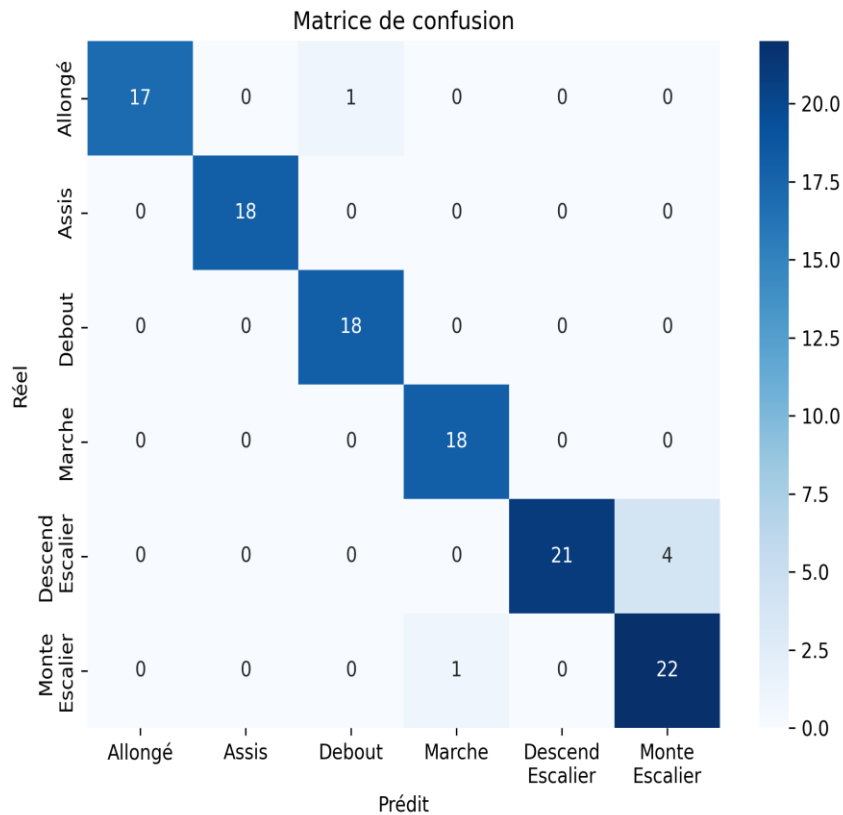
Concernant la sélection du classifieur optimal avec son groupe idéal de variables d'entrées, les résultats sont contenus dans *logs/2025-08-29_20-59-23*, *logs/2025-09-02_18-48-34* et *logs/2025-09-03_18-21-58*.

Lors de la première exécution, 100 jeux d'hyperparamètres parmi ceux présentés au 3.2 ont été testés. Les résultats obtenus sur les jeux ayant comme famille paramétrique les Transformers se sont avérés, en moyenne, nettement supérieurs à ceux ayant comme famille paramétrique CNN + LSTM et LSTM.

Ainsi, lors de la deuxième exécution, il a été joué 300 jeux d'hyperparamètres parmi ceux présentés au 3.2 mais en restreignant le choix de la famille paramétrique à celui des Transformers uniquement. On obtient alors comme meilleur résultat un classifieur ayant pour score de validation 93.6%+-3.8% et pour score de test 90.0%.

Sur chacune de ces deux premières exécutions, seule une sélection des variables par filtrage en amont a été effectuée, et une fois le meilleur jeu d'hyperparamètres déterminé, aucune sélection en avant des variables retenues n'a été effectuée. 26 variables ont été retenues via le processus de filtrage employé (voir 3.1).

Lors de la troisième exécution, on reprend le meilleur jeu d'hyperparamètres obtenu lors des deux premières exécutions et on effectue cette fois-ci, sur la base des 26 variables déjà retenues, une sélection en avant. On obtient alors comme résultat un classifieur nécessitant 7 variables d'entrées : $tBodyGyro-mean()-X$, $tBodyAcc-mad()-X$, $fBodyAccMag-mad()$, $angle(tBodyGyroMean,gravityMean)$, $fBodyAcc-bandsEnergy()-9,16$, $fBodyAccJerk-bandsEnergy()-1,16$, $fBodyAcc-bandsEnergy()-1,16$. Ce classifieur a pour score de validation 95.4%+-2.3% et pour score de test 95%.



Matrice de confusion obtenue sur les données de test

6. Conclusion

Ce projet m'a permis d'accroître significativement ma compréhension, ainsi que mes connaissances théoriques et pratiques, sur la méthodologie à mettre en place pour évaluer correctement la précision d'un modèle et sélectionner ses variables et hyperparamètres d'entraînement de façon optimale.